실전으로 배우는 빅데이터와 AI

2

스포츠 데이터 분석

(승률 계산부터 예측까지)

- **스포츠 데이터 분석**
  - 스포츠 경기에는 다양한 데이터가 존재합니다.
  - 경기 결과, 선수 기록, 팀 승률 등은 모두 데이터 분석을 통해 의미 있는 정보로 바뀔 수 있습니다.
  - 예를 들어 다음과 같은 질문을 데이터 분석으로 해결할 수 있습니다.
    - 우리 팀의 승률은 얼마나 될까?
    - 어떤 선수가 가장 좋은 성적을 기록하고 있을까?
    - 슈팅 수나 점유율이 경기 결과에 영향을 줄까?
    - 다음 경기에서 우리 팀이 이길 가능성은 얼마나 될까?
  - 이번 장에서는 스포츠 경기 데이터를 이용하여 데이터 분석의 기본 과정을 실습합니다

경기 데이터 구조 이해

- 데이터 분석을 시작하기 위해서는 먼저 데이터가 어떤 구조로 저장되어 있는지 이해해야 합니다.
- 스포츠 경기 데이터는 보통 다음과 같은 형태로 저장됩니다.

예시 데이터

date	opponent	home_ score	away_ score	shots	possession
2024-03-02	Central High	2	1	8	55
2024-03-09	Science High	1	2	6	48
2024-03-16	Future High	3	0	10	60
2024-03-23	Hanbit High	0	1	5	45

1. 경기 데이터 구조 이해

각 열(column)의 의미

Column	의미
date	경기 날짜
opponent	상대 팀
home_score	우리 팀 득점
away_score	상대 팀 득점
shots	우리 팀 슈팅 수
possession	볼 점유율

각 행(row)은 한 경기의 기록을 의미합니다.

CSV 데이터 만들기

- 이번 실습에서는 실제 데이터 분석과 유사한 환경을 만들기 위해 경기 데이터를 CSV 파일로 만들어 보겠습니다.
- 스포츠 경기 데이터는 보통 다음과 같은 형태로 저장됩니다.

① 파일 이름 : school_match_data.csv

② 데이터 내용

```
date,opponent,home_score,away_score,shots,possession
2024-03-02,Central High,2,1,8,55
2024-03-09,Science High,1,2,6,48
2024-03-16,Future High,3,0,10,60
2024-03-23,Hanbit High,0,1,5,45
2024-03-30,Ocean High,2,2,7,52
2024-04-06,Global High,4,1,11,63
2024-04-13,Sunrise High,1,0,6,50
2024-04-20,River High,3,2,9,57
2024-04-27,Tech High,2,3,8,49
2024-05-04,Creative High,5,1,12,65
2024-05-11,Future High,1,1,6,53
2024-05-18,Science High,2,0,7,56
```

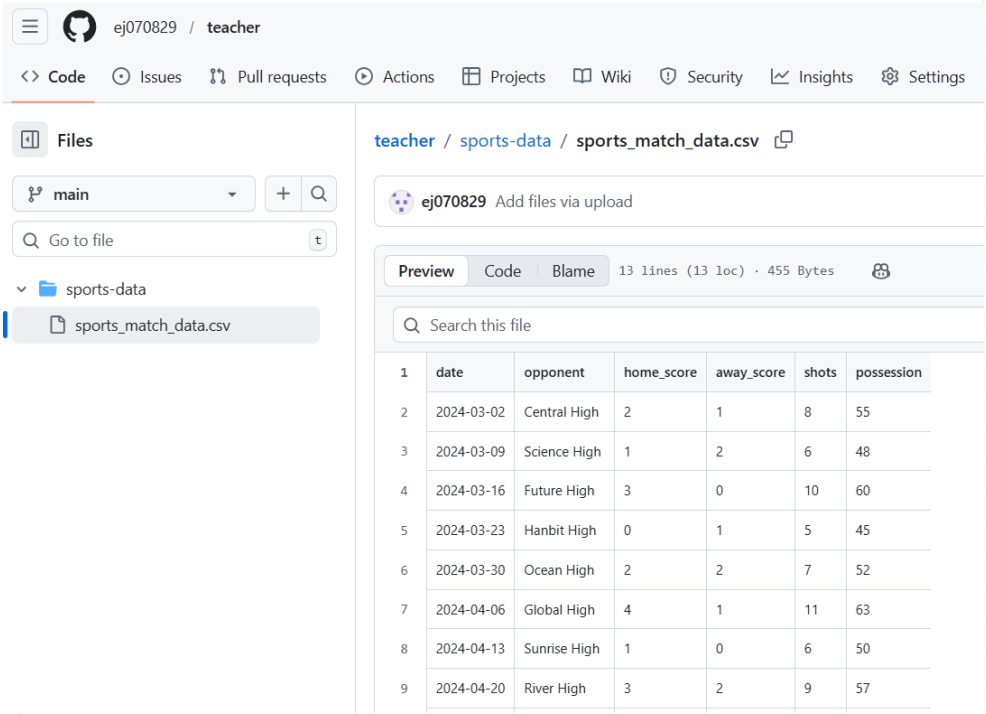
2. CSV 데이터 만들기

CSV 파일은 다음과 같이 만들 수 있습니다.

[방법 1] 메모장 또는 VS Code에서 작성한 후 저장

[방법 2] GitHub에서 다운로드

- github.com/ej070829/teacher/sports-data



1	date	opponent	home_score	away_score	shots	possession
2	2024-03-02	Central High	2	1	8	55
3	2024-03-09	Science High	1	2	6	48
4	2024-03-16	Future High	3	0	10	60
5	2024-03-23	Hanbit High	0	1	5	45
6	2024-03-30	Ocean High	2	2	7	52
7	2024-04-06	Global High	4	1	11	63
8	2024-04-13	Sunrise High	1	0	6	50
9	2024-04-20	River High	3	2	9	57

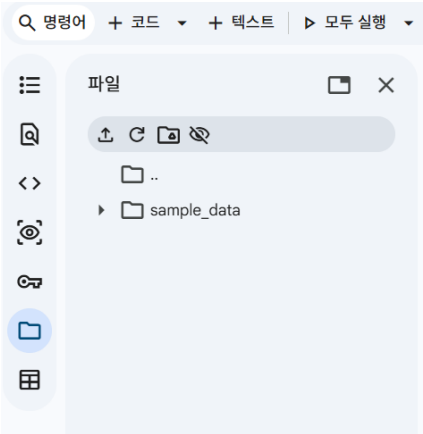
3. Pandas로 데이터 불러오기

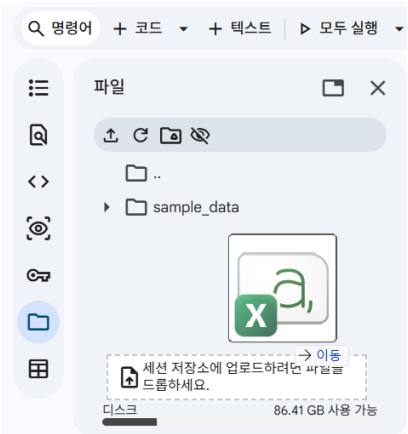
pandas로 데이터 불러오기

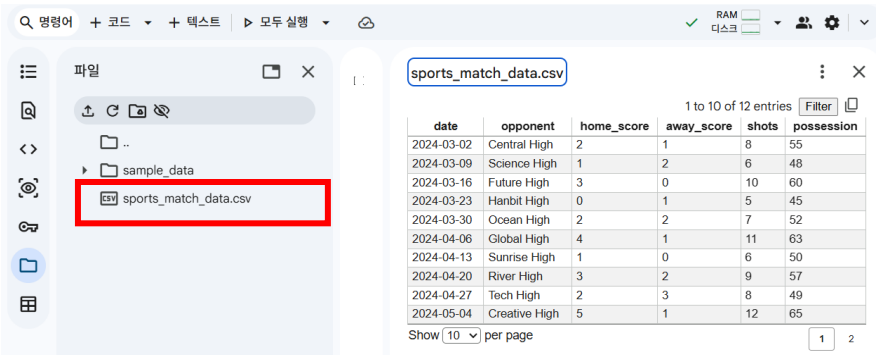
- 이제 pandas 라이브러리를 이용하여 데이터를 불러와 보겠습니다.
- pandas는 파이썬에서 데이터를 쉽게 읽고 정리하며 분석할 수 있도록 도와주는 데이터 분석 라이브러리입니다. CSV 파일, 엑셀 파일 등 다양한 데이터를 표 형태(DataFrame)로 처리할 수 있습니다.

① 파일 이름 : school_match_data.csv

② 데이터 내용 : school_match_data.csv파일을 코랩의 파일로 드레그







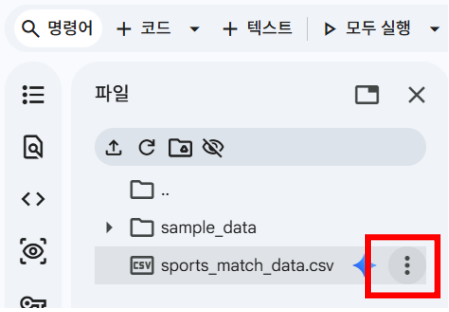
date	opponent	home_score	away_score	shots	possession
2024-03-02	Central High	2	1	8	55
2024-03-09	Science High	1	2	6	48
2024-03-16	Future High	3	0	10	60
2024-03-23	Hanbit High	0	1	5	45
2024-03-30	Ocean High	2	2	7	52
2024-04-06	Global High	4	1	11	63
2024-04-13	Sunrise High	1	0	6	50
2024-04-20	River High	3	2	9	57
2024-04-27	Tech High	2	3	8	49
2024-05-04	Creative High	5	1	12	65

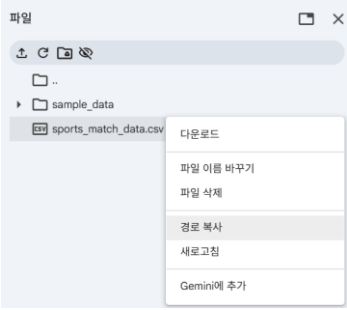
③ 코드 작성

```
import pandas as pd
df = pd.read_csv("/content/sports_match_data.csv")
print(df)
```

- import pandas as pd : pandas 라이브러리를 불러옵니다. pd는 pandas를 간단히 사용하기 위한 별칭입니다.
- pd.read_csv("/content/sports_match_data.csv") : CSV 파일을 읽어 데이터로 불러옵니다.

2. CSV 데이터 만들기





- df : 불러온 데이터를 저장하는 변수이며, DataFrame 형태로 저장됩니다.
- print(df) : 데이터 전체 내용을 화면에 출력합니다.

④ 코랩 코드 실행 결과

```
import pandas as pd

df = pd.read_csv("/content/sports_match_data.csv")

print(df)
```

```
           date       opponent  home_score  away_score  shots  possession
0    2024-03-02   Central High           2           1      8          55
1    2024-03-09   Science High           1           2      6          48
2    2024-03-16    Future High           3           0     10          60
3    2024-03-23    Hanbit High           0           1      5          45
4    2024-03-30     Ocean High           2           2      7          52
5    2024-04-06    Global High           4           1     11          63
6    2024-04-13   Sunrise High           1           0      6          50
7    2024-04-20     River High           3           2      9          57
8    2024-04-27      Tech High           2           3      8          49
9    2024-05-04  Creative High           5           1     12          65
10   2024-05-11    Future High           1           1      6          53
11   2024-05-18   Science High           2           0      7          56
```

- 실행 결과 중 데이터의 상위 5개 행만 보고 싶다면 다음 코드를 사용합니다.

```
print(df.head())
```

```
import pandas as pd
df = pd.read_csv("/content/sports_match_data.csv")
print(df.head())
```

```
           date       opponent  home_score  away_score  shots  possession
0    2024-03-02   Central High           2           1      8          55
1    2024-03-09   Science High           1           2      6          48
2    2024-03-16    Future High           3           0     10          60
3    2024-03-23    Hanbit High           0           1      5          45
4    2024-03-30     Ocean High           2           2      7          52
```

4. 팀 승률 계산하기

팀 승률 계산

- 경기 결과를 이용하여 우리 팀의 승률을 계산해 보겠습니다.
- 승률 계산 공식

승률 = 승리 경기 수 / 전체 경기 수

① 먼저 각 경기의 승패를 계산합니다.

- 코드 작성

```
df["win"] = df["home_score"] > df["away_score"]
print(df)
```

- 코랩 코드 실행 결과

```
df["win"] = df["home_score"] > df["away_score"]
print(df)
```

```
           date       opponent  home_score  away_score  shots  possession  ₩
0   2024-03-02   Central High           2           1      8          55
1   2024-03-09   Science High           1           2      6          48
2   2024-03-16    Future High           3           0     10          60
3   2024-03-23    Hanbit High           0           1      5          45
4   2024-03-30     Ocean High           2           2      7          52
5   2024-04-06    Global High           4           1     11          63
6   2024-04-13   Sunrise High           1           0      6          50
7   2024-04-20     River High           3           2      9          57
8   2024-04-27      Tech High           2           3      8          49
9   2024-05-04  Creative High           5           1     12          65
10  2024-05-11    Future High           1           1      6          53
11  2024-05-18   Science High           2           0      7          56

      win
0    True
1   False
2    True
3   False
4   False
5    True
6    True
7    True
8   False
9    True
10  False
11   True
```

- 코드 설명
  - df["home_score"] : 우리 팀의 득점 데이터를 가져옵니다.
  - df["away_score"] : 상대 팀의 득점 데이터를 가져옵니다.
  - df["home_score"] > df["away_score"] : 우리 팀 득점이 상대 팀 득점보다 큰지 비교하여 경기 결과를 판단합니다.
  - df["win"] = ... : 비교 결과를 win이라는 새로운 열(column)에 저장합니다.

4. 팀 승률 계산하기

② 승률 계산

- 코드 작성

```
win_rate = df["win"].mean()
print("우리 팀 승률:", win_rate)
```

- 코드 설명
  - df["win"] : 결기 승리 여부 결과가 저장된 win 열 데이터를 가져옵니다.
  - .mean( ) : 데이터의 평균값을 계산합니다.

- 코드 결과

```
win_rate = df["win"].mean()
print("우리 팀 승률:", win_rate)
```

```
우리 팀 승률: 0.5833333333333334
```

승률 계산은 58%임을 확인할 수 있습니다.

5. 축구 선수 성적 비교하기

축구 선수 성적 비교

– 이번에는 축구 선수 데이터를 분석해 보겠습니다.

① 파일 이름 : player_stats.csv

② 데이터 내용

```
player,goals,assists,shots,minutes
Minsoo,5,3,20,540
Jihoon,2,4,12,500
Hyunwoo,7,1,25,560
Junho,3,2,15,480
Seungmin,4,3,18,520
Taeyang,1,2,10,450
```

- 선수 데이터를 불러오겠습니다.

① 코드 작성

```
players = pd.read_csv("player_stats.csv")
print(players)
```

② 코드 설명

- pd.read_csv("player_stats.csv") : player_stats.csv 파일에 저장된 선수 데이터를 읽어옵니다.
- players = ... : 읽어온 데이터를 players라는 변수에 저장합니다. 이 데이터는 표 형태 데이터인 DataFrame으로 저장됩니다

③ 코드 결과

```
players = pd.read_csv("player_stats.csv")

print(players)
```

```
...      player  goals  assists  shots  minutes
0    Minsoo      5        3     20      540
1    Jihoon      2        4     12      500
2   Hyunwoo      7        1     25      560
3     Junho      3        2     15      480
4  Seungmin      4        3     18      520
5   Taeyang      1        2     10      450
```

5. 축구 선수 성적 비교하기

- 공격 포인트를 계산합니다.

  축구에서는 보통 골과 어시스트를 더한 값을 공격 포인트라고 합니다.

  ① 코드 작성

```
players["attack_point"]
        = players["goals"] + players["assists"]
print(players)
```

  ② 코드 설명

  - players["goals"] : 선수의 골 수 데이터를 가져옵니다.
  - players["assists"] : 선수의 어시스트 데이터를 가져옵니다.
  - players["goals"] + players["assists"] : 각 선수의 골과 어시스트를 더하여 공격 포인트를 계산합니다.
  - players["attack_point"] = ... : 계산된 값을 attack_point라는 새로운 열(column)에 저장합니다.

  ③ 코드 결과

```
players["attack_point"] = players["goals"] + players["assists"]

print(players)
```

```
...      player  goals  assists  shots  minutes  attack_point
0    Minsoo      5        3     20      540             8
1    Jihoon      2        4     12      500             6
2   Hyunwoo      7        1     25      560             8
3     Junho      3        2     15      480             5
4  Seungmin      4        3     18      520             7
5   Taeyang      1        2     10      450             3
```

- 가장 많은 공격 포인트를 가진 선수를 찾아봅니다.

  ① 코드 작성

```
top_player = players.sort_values("attack_point",
                ascending=False)
print(top_player)
```

  ② 코드 설명

  - players.sort_values("attack_point", ascending=False) : attack_point 값을 기준으로 데이터를 정렬합니다.

5. 축구 선수 성적 비교하기

- ascending=False : 값이 큰 순서(내림차순)로 정렬합니다.
- top_player = ... : 정렬된 결과를 top_player라는 변수에 저장합니다.

③ 코드 결과

```
top_player = players.sort_values("attack_point", ascending=False)

print(top_player)
```

```
      player  goals  assists  shots  minutes  attack_point
0     Minsoo      5        3     20      540             8
2   Hyunwoo       7        1     25      560             8
4  Seungmin       4        3     18      520             7
1    Jihoon       2        4     12      500             6
3     Junho       3        2     15      480             5
5   Taeyang       1        2     10      450             3
```

6. 데이터 탐색하기

- 데이터 분석에서는 데이터를 다양한 방식으로 탐색합니다.
- 예를 들어 다음과 같은 질문을 할 수 있습니다
  - 가장 많은 슈팅을 한 경기는 언제인가?
  - 점유율 평균은 얼마인가?
- 슈팅 수 기준 정렬하기

① 코드 작성

```
df_sorted = df.sort_values("shots", ascending=False)
print(df_sorted)
```

② 코드 결과

```
df_sorted = df.sort_values("shots", ascending=False)

print(df_sorted)
```

```
...          date       opponent  home_score  away_score  shots  possession  ₩
9   2024-05-04  Creative High           5           1     12          65
5   2024-04-06    Global High           4           1     11          63
2   2024-03-16    Future High           3           0     10          60
7   2024-04-20     River High           3           2      9          57
0   2024-03-02   Central High           2           1      8          55
8   2024-04-27      Tech High           2           3      8          49
11  2024-05-18   Science High           2           0      7          56
4   2024-03-30     Ocean High           2           2      7          52
6   2024-04-13   Sunrise High           1           0      6          50
1   2024-03-09   Science High           1           2      6          48
10  2024-05-11    Future High           1           1      6          53
3   2024-03-23    Hanbit High           0           1      5          45

      win
9    True
5    True
2    True
7    True
0    True
8   False
11   True
4   False
6    True
1   False
10  False
3   False
```

6. 데이터 탐색하기

- 점유율 평균
  - 점유율 데이터인 possession 데이터를 이용하여 전체 경기의 평균 점유율을 계산합니다.
  - 점유율 평균은 전체 경기에서 우리 팀이 공을 얼마나 많이 소유했는지를 나타내는 값입니다.
  - mean() 함수를 사용하면 모든 경기의 점유율 데이터를 평균으로 계산할 수 있습니다.

① 코드 작성

```
print("평균 점유율:", df["possession"].mean())
```

② 코드 설명

- df["possession"] : 경기 데이터에서 점유율(possession) 열의 데이터를 가져옵니다.
- .mean() : 해당 열에 있는 모든 점유율 값의 평균을 계산합니다.

③ 코드 결과

```
print("평균 점유율:", df["possession"].mean())
```

```
평균 점유율: 54.416666666666664
```

- **간단한 승패 예측 모델**
  - 간단한 규칙을 이용하여 다음 경기 결과를 예측해 보겠습니다.
  - 가정 : 평균 득점이 평균 실점보다 높으면 팀의 공격력이 더 강하다고 보고, 다음 경기에서 승리할 가능성이 높다고 가정합니다.

    평균 득점 > 평균 실점 → 승리 가능성 높음

  - 이 가정은 간단한 예측 모델을 만들기 위한 기준입니다.
  - 실제 스포츠 분석에서는 더 많은 데이터와 다양한 요소(선수 상태, 상대 팀 전력 등)를 고려하여 더 정확한 예측 모델을 만들어야 합니다.
  - 평균 득점 계산

① 코드 작성

```
avg_score = df["home_score"].mean()
avg_concede = df["away_score"].mean()

print("평균 득점:", avg_score)
print("평균 실점:", avg_concede)
```

② 코드 설명

- df["home_score"] : 경기 데이터에서 우리 팀의 득점(home_score) 데이터를 가져옵니다.
- df["away_score"] : 경기 데이터에서 상대 팀의 득점(실점) 데이터를 가져옵니다.

③ 코드 결과

```
avg_score = df["home_score"].mean()
avg_concede = df["away_score"].mean()

print("평균 득점:", avg_score)
print("평균 실점:", avg_concede)
```

```
평균 득점: 2.1666666666666665
평균 실점: 1.1666666666666667
```

경기 승패 예측 모델

- 경기 결과 예측
  - 우리는 평균 득점이 평균 실점보다 높으면 팀의 공격력이 더 강하다고 예측하기로 했습니다.

① 코드 작성

```
if avg_score > avg_concede:
    print("다음 경기 승리 가능성이 높습니다.")
else:
    print("다음 경기 패배 가능성이 있습니다.")
```

② 코드 결과

```
avg_score = df["home_score"].mean()
avg_concede = df["away_score"].mean()

print("평균 득점:", avg_score)
print("평균 실점:", avg_concede)
```

```
평균 득점: 2.1666666666666665
평균 실점: 1.1666666666666667
```

- 이와 같은 방식은 기초적인 예측 모델입니다.
- 실제 스포츠 분석에서는 머신러닝 모델을 사용하기도 합니다.

프로젝트 과제

- **다음 질문을 코드를 사용하여 과제를 해결해 보세요.**

① 우리 팀의 승률은 얼마인가?

② 가장 많은 골을 넣은 선수는 누구인가?

③ 공격 포인트가 가장 높은 선수는 누구인가?

④ 슈팅 수가 가장 많았던 경기는 언제인가?

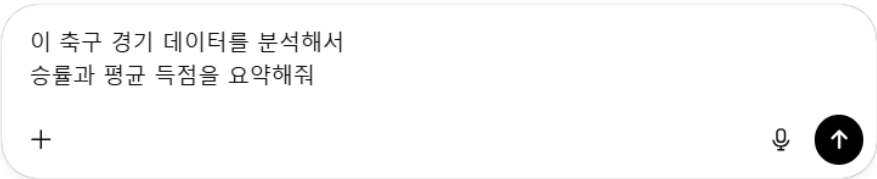
⑤ 다음 경기 승패는 어떻게 예측되는가?

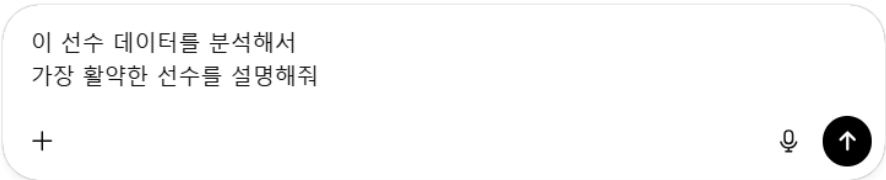
- **GPT를 활용하여 다음 질문을 해결해 보세요.**

- AI 도구를 활용하면 데이터 분석을 더 쉽게 수행할 수 있습니다. 예를 들어 다음과 같은 질문을 GPT에게 해보세요.

① GPT를 활용하여 경기 데이터 요약하기

다음과 같이 GPT에게 질문해보세요.



② GPT를 활용하여 선수 분석 설명 생성하기

다음과 같이 GPT에게 질문해보세요.